

计算机网络第三章 数据链路层 考试复习总结

2026-05-09

目录

1 数据链路层概述 (3.1 设计问题)	2
2 差错检测与纠错 (3.2)	4
3 基本数据链路层协议 (3.3)	6
4 滑动窗口协议 (3.4)	8
5 数据链路协议实例 (3.5)	10
6 核心考点速记	13

1 数据链路层概述（3.1 设计问题）

在协议栈中的位置 [p.3]

数据链路层（Data Link Layer, DLL）位于物理层之上、网络层之下。

- 向下：利用物理层提供的位流服务 [p.3]
- 向上：向网络层提供明确的服务接口 [p.3]
- 局域网的数据链路层分为两个子层：
 - LLC（Logical Link Control, 逻辑链路控制）
 - MAC（Media Access Control, 介质访问控制）

数据链路层的功能 [p.4]

- (1) 成帧（**Framing**）：将比特流划分成“帧”，主要目的是检测和纠正物理层传输中可能出现的错误 [p.4]
- (2) 差错控制（**Error Control**）：处理传输中出现的差错，如位错误、丢失等 [p.4]
- (3) 流量控制（**Flow Control**）：确保发送方的发送速率不大于接收方的处理速率，避免接收缓冲区溢出 [p.4]

数据链路层提供的服务 [p.5]

服务类型	确认	连接	适用场景
无确认无连接 (Unacknowledged Connectionless)	否	否	误码率低的可靠信道, 实时通信; 例: 以太网
有确认无连接 (Acknowledged Connectionless)	是	否	不可靠信道 (无线); 例: 802.11 WiFi
有确认有连接 (Acknowledged Connection-oriented)	是	是	长延迟的不可靠信道

成帧（Framing）方法 [p.6–17]

关键问题：如何标识一帧的开始和结束？即帧同步/帧定界问题 [p.7]。

1. 字节计数法 (Byte Count) [p.8–9]

在帧头部用一个计数字节标明帧的长度。

优点：简单

致命缺点：如果计数字节出错，会破坏帧的边界，导致一连串帧的错误 [p.9]

2. 带字节填充的定界符法 (Flag bytes with byte stuffing) [p.10–13]

- 定界符 FLAG = 01111110 (0x7E)，用于区分帧的边界 [p.10]
- 发送方：检查有效载荷，若出现 FLAG 则在其前插入转义字节 ESC；若出现 ESC 则在其前再插入一个 ESC [p.11]

- **接收方**：逐个检查每个字节：收到 ESC 则后一字节无条件成为有效载荷；收到 FLAG 则为帧的边界 [p.12]
- **不足**：依赖于 8 位字节编码，不适用于所有编码体系 [p.13]

3. 带比特填充的定界符法 (Flag bits with bit stuffing) [p.14–16]

定界符为 01111110 (0x7E, 即两个 0 之间连续 6 个 1) [p.14]。

- **发送方**：若在有效载荷中出现连续 5 个 1 比特，则直接插入 1 个 0 比特 [p.15]
- **接收方**：若出现连续 5 个 1 比特：
 - 下一比特为 0：为填充，直接丢弃 0 比特 [p.16]
 - 下一比特为 1：连同后一比特的 0，构成定界符，帧结束 [p.16]

4. 物理层编码违例 (Physical Layer Coding Violations) [p.17]

核心思想：选择的定界符不会在数据部分出现。

- **4B/5B 编码方案**：4 比特数据映射成 5 比特编码，剩余的一半码字 (16 个) 未使用，可用作定界符 [p.17]
- **传统以太网/802.11**：长前导码 (preamble) 可作定界符 [p.17]
- **曼彻斯特/差分曼彻斯特编码**：持续的高电平 (或低电平) 为违例码，可用作定界符；例如 802.5 令牌环网 [p.17]

差错控制 (Error Control) [p.18]

信道噪声导致的数据传输问题：

- **差错 (Incorrect)**：数据发生错误
- **丢失 (Lost)**：接收方未收到
- **乱序 (Out of order)**：先发后到
- **重复 (Repeated delivery)**：一次发送，多次接收

解决方案：

- **确认 (Acknowledgment)**：接收方校验数据，给发送方应答，防止差错 [p.18]
- **定时器 (Timer)**：发送方启动定时器，防止丢失 [p.18]
- **序号 (Sequence Number)**：接收方检查序号，防止乱序和重复交付 [p.18]

流量控制 (Flow Control) [p.19]

- 接收方的处理速率有限，接收缓冲区可能溢出 [p.19]
- **基于反馈 (Feedback-based)** 的流量控制：接收方反馈，发送方调整发送速率 [p.19]
- **基于速率 (Rate-based)** 的流量控制：发送方根据内置机制自行限速 [p.19]

2 差错检测与纠错 (3.2)

差错产生原因 [p.55]

- **随机热噪声 (Random Noises):** 信道本身的热噪声, 导致**随机错误** (某位错), 可通过提高信噪比等方法抑制 [p.55]
- **冲击噪声 (Burst Noises):** 外界引起的冲击噪声, 导致**突发错误** (一连串码元均出错) [p.55]
- **突发长度:** 从突发错误发生的第一个出错码元到最后一个出错码元间所有码元的个数 [p.55]

错误分类 [p.56–57]

- 一位错 (Single-bit error)
- 多位错 (Multi-bit error): 2 到多个位错
- 突发错误 (Burst error): 2 到多个位突发错误

误码率 (BER, Bit Error Rate) [p.58]

$$BER = P_e = \frac{\text{发生差错的码元数}}{\text{接收的总码元数}}$$

计算机网络中误码率一般要求低于 10^{-6} 。

码字基本概念 [p.62]

- **数据位 (Data bits):** 要传输的数据 (信息位)
- **冗余位 (Redundant bits):** 用于差错检测的附加位
- **码字 (Codeword):** 数据位 + 冗余位 = 要发送的数据

检错码与纠错码 [p.64–66]

特征	检错码 (Error-Detecting Code)	纠错码 (Error-Correcting Code)
功能	推断是否有错, 不能定位	判断并纠正错误 (定位出错位)
冗余量	较小	较大
适用信道	高可靠、误码率低的信道 (如光纤)	错误频繁的信道 (如无线链路)
处理方式	请求重传	前向纠错 (FEC)
应用层次	链路层、网络层、传输层	物理层、更高层 (实时流媒体)

常用检错方法 [p.67]

- **VRC (Vertical Redundancy Check):** 垂直奇偶校验
- **LRC (Longitudinal Redundancy Check):** 水平奇偶校验
- **CRC (Cyclic Redundancy Check):** 循环冗余校验
- **Checksum:** 校验和

奇偶校验 (Parity Check) [p.68–72]

通过增加冗余位使码字中“1”的个数保持奇数 (奇校验) 或偶数 (偶校验)。简单经济, 但漏检率较高 [p.68]。

- **垂直奇偶校验 (VRC)** [p.69–70]: 每字符加一个校验位。可检测所有 1 位错, 但只能检测出错数为奇数的多位错/突发错。漏检率 $\approx 1/2$ 。编码效率 $R = p/(p+1)$
- **水平奇偶校验 (LRC)** [p.71]: 对一组字符的对应位进行校验。漏检率 $< 1/2$, 实现更复杂
- **水平垂直奇偶校验 (LRC+VRC)** [p.71–72]: 误码率可减少到原来的 $1/100 \sim 1/10000$, 但若某信息段中出现偶数个差错且另一信息段对应位置也出现偶数个差错, 则无法检测 [p.72]

循环冗余校验码 (CRC) [p.74–77]

又称多项式码 (Polynomial Code), 是一种线性分组码, 基于严格的代数学理论基础, 检纠错能力非常强, 广泛应用于计算机网络和数据通信。

编码步骤 [p.75]:

1. 若生成多项式 $G(x)$ 的阶是 r , 将信息位左移 r 位, 得 $x^r M(x)$

2. 作模 2 除法, 求余数 $r(x)$:

$$\frac{x^r M(x)}{G(x)} = Q(x) + \frac{r(x)}{G(x)}$$

3. 码字 $T(x) = x^r M(x) + r(x)$

标准多项式 [p.77]:

- CRC-16 / CRC-ITU: 可查出全部一位错、双位错、奇数位错, 全部 16 位及以下突发错, 99.997% 的 17 位突发错及 99.998% 的 18 位或更长突发错
- 传输速率为 9600bps 时, 数据传输 3000 年才会有一个差错漏检 [p.77]

校验和 (Checksum) [p.78–79]

将数据看成二进制整数序列, 划分为规定长度 (8/16/32 位), 计算它们的和 (有进位则加到校验和中), 接收端重新计算并比较。算法简单但检错强度较弱 [p.78]。

TCP/IP 校验和 [p.79]: 发送方进行 16 位二进制补码求和, 结果取反后随数据发送; 接收方进行 16 位补码求和 (含校验和), 结果若非全 1 则检测到错误。

典型纠错码 [p.81]

- 海明码 (Hamming Code)
- 二进制卷积码 (Binary Convolutional Code)
- 里德-所罗门码 (Reed-Solomon Code)
- 低密度奇偶校验码 (LDPC)

海明码 (Hamming Code) [p.82–88]

冗余位计算 [p.82] : 设数据 m 位, 冗余位 r 位, 则:

$$2^r \geq m + r + 1$$

其中 1 种状态表示无差错, $m + r$ 种状态分别表示每一位发生的差错。

例: $m = 7$ (ASCII), 最小 $r = 4$, 因为 $2^4 \geq 7 + 4 + 1$ (即 $16 \geq 12$)。

冗余位定位 [p.83–84] : 冗余位放在 2^n 位置 (第 1, 2, 4, 8 位), 每个 r 位是一组数据位的奇偶校验码:

- r_1 : 第 1, 3, 5, 7, 9, 11 位
- r_2 : 第 2, 3, 6, 7, 10, 11 位
- r_3 : 第 4, 5, 6, 7 位 (如果是 12 位则还有 8-11 位)
- r_4 : 第 8, 9, 10, 11 位

检错与纠错 [p.88] : 接收端重新计算奇偶校验码, 按 $r_4 r_3 r_2 r_1$ 顺序组成二进制数, 该数值即为出错位位置。将该位反转即可纠错。

3 基本数据链路层协议 (3.3)

实现位置 [p.21]

物理层进程和部分数据链路层进程运行在专用硬件 (网卡) 上, 其余部分和网络层运行在 CPU 上作为操作系统的一部分 (设备驱动)。

协议定义与事件 [p.22]

基本事件类型: 帧到达 (frame_arrival)、帧出错 (cksum_err)、超时 (timeout)。

乌托邦式单工协议 (Utopian Simplex Protocol) [p.23–24]

假设 [p.23]:

- 单工 (Simplex): 数据单向传输
- 完美信道: 帧不会丢失或受损
- 始终就绪: 发送方/接收方网络层始终就绪
- 瞬间完成: 发送方/接收方能生成/处理无穷多的数据

不处理任何流量控制或纠错, 接近无确认无连接服务, 必须依赖更高层解决问题 [p.23]。

无错信道上的停等式协议 [p.25–27]

不再假设接收方能无限高速处理数据。发送方以高于接收方能处理的速度发送帧，导致接收方被“淹没” (overwhelming)。仍假设信道不出错 [p.25]。

停等协议 (Stop-and-Wait) [p.26]:

- 发送方发一帧后暂停，等待确认 (Acknowledgment) 到达后发下一帧
- 接收方完成接收后，回复确认。确认帧内容是哑帧 (dummy frame)
- 数据单向传输，但需要双向传输链路 (半双工物理信道)

有错信道上的单工停等式协议 [p.28–34]

假设信道可能出错：帧可能被损坏或丢失 [p.28]。

简单解决方案：发送方增加一个计时器 (timer)，如果一段时间没有收到确认就超时重发 [p.28]。

ARQ/PAR 机制 [p.29–30]

引入序号 (**SEQ: Sequence Number**) 来区分新帧和重复帧 [p.30]:

- 接收方需要区分到达的帧是第一次发来的新帧，还是应该丢弃的重复帧
- 在帧头部放一个序号，接收方检查序号判断
- 此协议中最小序号位数：1 bit 序号 (0 或 1) 就足够——因为只有当前帧和直接后续帧之间可能不明确

ARQ (Automatic Repeat reQuest) = 自动重复请求 (带有重传的肯定确认)，也叫 **PAR** (Positive Acknowledgment with Retransmission) [p.30]。

协议规则 [p.31]

- **发送方**：初始化帧序号为 0。发送帧后等待——正确确认则发下一帧；超时/错误确认则重发
- **接收方**：初始化期待 0 号帧。收到正确帧则交给网络层并发送确认；收到错误帧则发送上一成功接收帧的确认

效率评估 [p.32]

设 F = 帧大小 (bits)， R = 信道带宽 (bits/sec)， I = 传播延迟 + 处理时间 (sec):

- 每帧发送时间: $T_{\text{frame}} = F/R$
- 总延迟: $D = 2I$ (往返)
- 停止等待协议工作时间 F/R ，空闲时间 D

信道利用率:

$$\text{Line Utilization} = \frac{F}{F + R \cdot D}$$

当 $F/R < D$ 时，利用率 $< 50\%$ [p.32]。

长肥网络 (LFN) [p.33]

若网络的带宽-延迟乘积明显大于 10^5 bits (~ 12 KB), 则被认为是长肥网络。

例: $F = 1000$ bits, $R = 1$ Mbps, 传播延迟 = 270 ms。每帧发送时间 = 1 ms, 由于传播延迟长, 发送者在 541 ms 之后才能收到确认, 信道利用率 $\approx 1/541$ (极低) [p.34]。

提高效率的方法 [p.34]: 使用更大的帧, 但帧长度受 BER 限制 (帧越大, 出错概率越高, 导致更多重传)。

4 滑动窗口协议 (3.4)

停等协议的性能问题 [p.36]

$$\text{线路效率} = \frac{F}{F + R \cdot D}$$

在传输延迟较长的信道上, 停等协议无法使数据充满管道, 信道利用率很低。

解决办法: 流水线协议/管道协议——允许发送方在没收到确认前连续发送多个帧 [p.36]。

滑动窗口基本思想 [p.37–42]

发送方和接收方都具有一定的缓冲区 (即窗口), 发送端在收到确认前可发送多个帧 [p.37]。

核心概念 [p.39]:

- **发送窗口:** 连续允许发送的帧序号范围, 大小记作 W_T , 表示未确认前可连续发出的最多数据帧数
- **接收窗口:** 连续允许接收的帧序号范围, 大小记作 W_R
- 发送窗口和接收窗口的序号上下界不一定相同, 大小也可以不同

滑动窗口机制要点 [p.41]:

- 对可连续发出的最多帧数 (已发但未确认) 作限制
- 序号循环重复使用有限的帧序号
- 流量控制: 接收窗口驱动发送窗口的转动
- **累计确认 (Cumulative ACK):** 不必对收到的分组逐个发送确认, 而是对按序到达的最后一个分组发送确认

回退 N 协议 (Go-Back-N, GBN) [p.43–47]

原理 [p.44]: 允许发送方发送多个分组而不用等待确认。正常情况下, 发送、接收窗口会随确认状态随包右移。

数据包丢失时的行为 [p.45]:

- 当发送过程中的 2 号包丢失时: 发送方没有收到 ACK2, 后面 3、4 号包的确认全部变成 ACK1 (因接收方丢弃 3 和 4), 全部返回 ACK1

- 经过一段时间后，定时器超时确认没收到 ACK3/ACK4，发送方将重新发送 2、3、4 号所有未确认包

ACK 丢失时的行为 [p.46]:

- 假设 ACK2 丢失：如接收方没有收到分组 2，则后面返回的都是 ACK1
- 因本次返回的为 ACK3, ACK4，所以发送方可判断接收方已接收到消息，不再进行重发

实现要点 [p.47]:

- 必须增加序号范围；发送方需缓存多个分组
- 发送窗口：发送方维持一组连续允许发送的帧序号
- 接收窗口：接收方维持一组连续允许接收帧序号（大小为 1）
- 发送方必须响应的三件事：
 1. 上层调用：检测有没有可用的序号，有就发送
 2. 收到 ACK：对 n 号帧的确认采用累积确认方式
 3. 超时事件：重传所有已发送未确认的分组

选择性重传协议 (Selective Repeat, SR) [p.48–52]

设计思想 [p.48]:

- 若发送方发出连续的若干帧后，收到对其中某一帧的否认帧或某一帧的定时器超时，则只重传该出错帧
- 接收窗口大小 > 1 ：暂存接收窗口中序号在出错帧之后的数据帧
- 优点：避免重传已正确传送的帧
- 缺点：接收端需占用一定容量的缓存

数据包丢失时的行为 [p.51]: 与 GBN 不同，接收方在没收到分组 2 的情况下依然返回 ACK3, ACK4。ACK1 返回后，分组 5、6 即可发送。接收方缓存分组 3-6，等待分组 2 的定时器超时后重新发送 2。

ACK 丢失时的行为 [p.52]: 分组 2 会因为 ACK2 被丢失后在定时器到时后重新发送一次。如果在接收过程中有丢失发生，SR 的效率不如 GBN [p.52]。

最大窗口数 [p.53]

滑动窗口协议中，发送窗口和接收窗口的大小关系与序号空间有关。

协议	发送窗口大小	接收窗口大小
停等协议	1	1
Go-Back-N	$W_T > 1$	$W_R = 1$
Selective Repeat	$W_T > 1$	$W_R > 1$

5 数据链路协议实例 (3.5)

PPP 协议简介 [p.91]

PPP (Point-to-Point Protocol) 由 IETF 制定, 1994 年成为正式标准 (RFC 1661)。

- 目前使用最多的数据链路层协议之一
- 能够在不同的链路上运行
- 能够承载不同的网络层分组
- 特点: 简单、灵活

PPP 三大组成部分 [p.92]

1. **成帧方法**: 明确定界一个帧的结束和下一帧的开始, 也允许进行错误检测
2. **链路控制协议 (LCP, Link Control Protocol)**: 负责启动线路、测试线路和选项协商, 并在不再需要时稳妥地释放
3. **网络控制协议 (NCP, Network Control Protocol)**: 协商网络层选项; 对每个支持的网络层协议有不同的 NCP

PPP 帧格式 [p.93–94]

Flag	Address	Control	Protocol	Payload	FCS
0x7E	0xFF	0x03	1-2 bytes	Variable	2/4 bytes

- Flag = 0x7E (01111110), 帧定界符 [p.93]
- Address = 0xFF (广播地址)
- Control = 0x03 (默认值)
- Protocol 字段: 标识载荷中的分组类型 (LCP, NCP, IP, IPX, AppleTalk 等), 首字节为 0 则是网络层协议 [p.93]
- 载荷长度可变, 默认 1500 字节 (LCP 协商前)
- FCS (Frame Check Sequence): 采用 32 位或 16 位 CRC [p.93]

协议字段值 [p.94]:

- 0x0021: IP 数据报
- 0x8021: 网络控制数据 (NCP)
- 0xC021: 链路控制数据 (LCP)

PPP 工作状态及转换 [p.95–103]

PPP 六种状态 [p.95–96]:

1. **链路静止 (Link Dead):** 起始和终止状态, 通信双方尚无链路。物理层可用时进入链路建立阶段 [p.97]
2. **链路建立 (Link Establishment):** LCP 通过交换配置包建立连接。LCP 配置选项包括: 最大帧长、鉴别协议规约、不使用地址和控制字段 [p.98]
3. **身份认证 (Authentication):** 可选, 默认不需要。若选择认证协议, 需在链接建立阶段请求使用 [p.100]
4. **网络连接 (Network):** 每个网络层协议必须由适当的 NCP 单独配置 [p.101]
5. **数据通信 (Data/Open):** 两个 PPP 端点可彼此发送分组或 Echo-* 帧 [p.102]
6. **链路终止 (Link Terminate):** 任何时候可终止。通过交换终止包关闭连接, 随后进入链路静止阶段 [p.103]

LCP 帧类型 [p.98–99]:

- 配置类: Configure-Request, Configure-Ack (全部接受), Configure-Nak (理解但不能接受), Configure-Reject (无法识别/不接受)
- 关闭类: Terminate-Request, Terminate-Ack
- 拒绝类: Code-Reject (收到未知请求), Protocol-Reject (被请求协议未知)
- 测试类: Echo-Request, Echo-Reply (测试线路质量)
- Discard-Request: 只需丢弃该帧 (用于测试)

PPPoE 概述 [p.105–115]

PPPoE (Point-to-Point Protocol over Ethernet) [p.106]:

- 提供在以太网链路上的 PPP 连接
- 实现传统以太网不能提供的身份验证、加密以及压缩等功能
- 实现基于用户的访问控制、计费、业务类型分类等
- 使用 Client/Server 模型, 服务器通常是接入服务器

PPPoE 与以太网对比 [p.105]:

- 以太网优点: 原理简单, 应用广, 设备成本低
- 以太网缺点: 安全性较低 (广播信道), 不宜管理, 无认证功能
- PPP 优点: 原理简单, 安全性高 (点对点信道), 提供认证机制, 良好的访问控制和计费功能

PPPoE 组网方式 [p.107–108]:

- 方式 1: 设备之间建立 PPP 会话, 所有主机通过同一 PPP 会话传送数据; 主机不需安装 PPPoE 客户端拨号软件
- 方式 2: PPP 会话建立在主机和运营商路由器之间, 为每个主机建立各自的 PPP 会话; 每个主机有独立账号, 运营商可对用户进行计费和控制

PPPoE 报文格式 [p.109–110]: PPPoE 报文封装在以太网帧中 (类型字段 = 0x8863 Discovery 阶段 / 0x8864 Session 阶段)。

PPPoE 报文头部字段:

- 版本号 (Version): 值为 0x1
- 类型 (Type): 值为 0x1
- 代码 (Code): 报文类型
- 会话 ID (Session ID): 标识会话
- 长度 (Length): 有效载荷长度

PPPoE 代码值 [p.110]:

- 0x00: 会话数据
- 0x09: PADI (PPPoE Active Discovery Initiation)
- 0x07: PADO (PPPoE Active Discovery Offer) 或 PADT
- 0x19: PADR (PPPoE Active Discovery Request)
- 0x65: PADS (PPPoE Active Discovery Session-confirmation)

PPPoE 的三阶段工作过程 [p.111–115]:**1. Discovery 阶段 [p.112–113]:**

- Client 广播 PADI 报文 (包含想要的服务类型信息)
- Server 比较请求服务与自身能提供的服务, 若可提供则单播回复 PADO 报文
- Client 可能收到多个 Server 的 PADO, 选择最先到达的对应 Server, 单播 PADR
- Server 生成唯一 Session ID, 发送 PADS 给 Client, 进入 Session 阶段

2. Session 阶段 (PPP 协商) [p.114]:

- LCP 阶段: 建立、配置和检测链路连接
- 认证阶段: 认证成功则进入 NCP
- NCP 阶段: 配置网络层协议 (常用 IPCP, 配置 IP 和 DNS 等)
- 数据传输阶段: PPP 报文传输

3. **Terminate** 阶段 [p.115]:

- 两种终止方式: PPP 协议自身 (终结报文) 结束会话; 使用 PADT 报文终结会话
- 进入 Session 阶段后, Client 和 Server 均可通过发送 PADT 报文结束连接
- PADT 可在会话建立后的任意时刻单播发送

6 核心考点速记

关键公式

1. 停等协议信道利用率:

$$U = \frac{F}{F + R \cdot D}$$

其中 F = 帧长 (bits), R = 带宽 (bps), D = 往返延迟 (s) [p.32, 36]

2. 海明码冗余位:

$$2^r \geq m + r + 1$$

其中 m = 数据位数, r = 冗余位数 [p.82]

3. **CRC** 编码:

$$T(x) = x^r M(x) + r(x), \quad r(x) = x^r M(x) \bmod G(x)$$

[p.75]

4. 垂直奇偶校验编码效率:

$$R = \frac{p}{p + 1}$$

其中 p = 信息块位数 [p.70]

5. 误码率:

$$\text{BER} = P_e = \frac{\text{出错的码元数}}{\text{总码元数}}$$

[p.58]

关键概念速记表

概念	核心要点
成帧方法	字节计数 (脆弱)、字节填充 (FLAG+ESC)、比特填充 (连续 5 个 1 插 0)、编码违例
差错控制	确认 + 定时器 + 序号 (SEQ)
流量控制	Feedback-based (接收方反馈) / Rate-based (发送方自限速)
CRC	多项式模 2 除法; CRC-16 可查所有 1/2/奇数位错和 16 位以下突发错
海明码	$2^r \geq m + r + 1$; 冗余位在 2^n 位置; 纠错时 r 码组成二进制定位出错位
停等协议	发一帧等确认; 利用率 $< 50\%$; 用 1-bit 序号即可
ARQ	发送方 + 计时器 + 序号; 超时重传
滑动窗口	允许连续发多帧, 不用等确认; 流水线/管道思想
Go-Back-N	$W_T > 1, W_R = 1$; 出错时退回 N 帧重发所有未确认包; 累积确认
Selective Repeat	$W_T > 1, W_R > 1$; 只重传出错帧; 接收端需缓存乱序帧
PPP	成帧 + LCP + NCP; Flag=0x7E; 六种状态转换; 支持认证
PPPoE	以太网上跑 PPP; Discovery($\rightarrow$ Session $\rightarrow$ Terminate); PADI/PADO/PADR/PADS/PADT

核心对比

特性	停等协议	GBN	SR
发送窗口	1	> 1	> 1
接收窗口	1	1	> 1
确认方式	逐一确认	累积确认	逐一确认
出错处理	超时重发一帧	退回 N 帧全部重发	只重传出错帧
接收缓存	不需要	不需要	需要缓存
效率	低	中	高 (但 ACK 丢失时不如 GBN)

— 祝考试顺利! —